6. TRD — TCC Event Clearance App

Nov 8, 2026 · @rogiie

A React and TypeScript PWA on Supabase, using device-held ECDSA keys so QR codes and stamps can be verified with no internet.

Technology stack

These are recommendations; confirm current versions and limits when you start.

Layer	Choice	Why
App	React and TypeScript, built with Vite, as a PWA with a Workbox service worker	One codebase for Android, iPhone and desktop; the app shell opens offline
Styling	Tailwind CSS	Responsive from phone to laptop
Local data	IndexedDB through Dexie	Rosters, scan queue and records offline
Cryptography	WebCrypto API, ECDSA P-256 with SHA-256	Built into browsers; no extra library
QR drawing and scanning	A QR generator library to draw; jsQR or zxing-wasm to scan	The browser's BarcodeDetector is missing on iPhone Safari
Face check	MediaPipe Face Detector, running on the device	Counts faces in the live camera frame
Signature pad	signature_pad library	Smooth finger strokes, exports PNG or SVG
Backend	Supabase: PostgreSQL, Auth (Google), Storage, Edge Functions	One service with row-level security
Card rendering	HTML and SVG drawn from layout_json; html-to-image and jsPDF for export	Real text, printable, same code on phone and dashboard
Excel export	ExcelJS	Sheets per department with formatting

Layer	Choice	Why
Hosting	Static host for the PWA (Cloudflare Pages, Vercel or Netlify) plus Supabase	HTTPS is required for camera and location
AI artwork	An image-generation API called from an Edge Function	The API key never reaches the browser

QR token and stamp specification

Keys. `generateKey({name:'ECDSA', namedCurve:'P-256'}, false, ['sign','verify'])` creates a non-extractable key pair on the student's phone and on each scanner phone. The key object is stored in IndexedDB. WebCrypto returns signatures as 64 raw bytes (r then s); the server must verify in the same format.

QR token (student to scanner)

Field	Meaning
v	Format version
sid	Student ID
eid	Event ID
w	Time window: unix seconds divided by 30, rounded down
n	Random 8-byte nonce
loc	Optional latitude and longitude, only when the event's location check is on
sig	ECDSA signature over the fields above (64 bytes)

The fields are packed compactly and encoded as base64url, giving a QR of roughly 150 characters. The phone creates a new token every 30 seconds.

Scanner checks, in order: version, event matches, student is in the roster and the device key is active, signature valid, window within one step of the scanner's window (plus the clock tolerance), no valid check-in yet for this student and slot, eligibility, then the location distance when present.

Stamp record (scanner signed): serial number, event, slot, student, venue, scanned_at, officer, scanner key and method, signed with the scanner's key. The serial is the scanner key ID plus an increasing counter, so it is unique without the server.

Receipt QR (optional): the stamp record and its signature, encoded the same way. The student's phone verifies it with the scanner's public key from the event pack and then draws the stamp.

PWA, install gate and offline storage

- **Manifest and service worker.** The manifest sets standalone display, icons and start URL. Workbox precaches the app shell so the app opens with no connection.
- **Install gate.** The app treats `matchMedia('(display-mode: standalone)')` or `navigator.standalone` as installed. Anything else shows only the install screen. Desktop-width screens skip the gate. This is a soft gate; the real security does not depend on it.
- **In-app browsers.** Detect Messenger and Facebook browsers from the user agent and show "Open in Safari or Chrome".
- **Android.** Capture `beforeinstallprompt` and trigger it from the "Install to continue" button.
- **iPhone.** No install API exists, so show the Share, then Add to Home Screen picture guide.
- **Storage.** Use IndexedDB and call `navigator.storage.persist()`. Show "ready for offline" only after the roster and event pack pass a checksum.
- **Sync.** Background Sync is not available on iPhone, so sync runs when the app opens, comes to the foreground or fires the `online` event. Uploads are idempotent through client_uuid.
- **Camera.** `getUserMedia` needs HTTPS. Torch and focus control are unreliable on iPhone, so rely on a large, high-contrast QR.
- **Location.** `navigator.geolocation` with high accuracy, a timeout and a retry button; distance is computed on the device with the haversine formula.
- **Clock.** Default tolerance of one 30-second window; when the scanner last synced it records its offset from server time and warns if a student's token is far outside it.

Face check, card rendering, AI artwork and exports

Face check. MediaPipe Face Detector runs on live camera frames. The photo is accepted only when exactly one face is found, it is large and centered, and the frame is bright and sharp enough. The exact thresholds are starting values to tune on real phones. The accepted photo is saved as a 480 px JPEG (about 40 KB) and a 160 px thumbnail (about 12 KB).

Card rendering. A single renderer draws the card from the event's `layout_json`. Element types are text, student field, image, logo, line, rectangle, QR and signature box. The same code produces the on-phone card, the dashboard print view and the PNG or PDF export. A signature box is filled from attendance records: the officer's signature image, the printed name and position, the time and the serial number.

AI artwork. An Edge Function receives the theme and event details, builds the prompt (always asking for no text in the image), calls the image API and stores three or four options. The SBO picks one; it is compressed to a WebP of 150 KB or less. A default limit of 5 generations per event applies, and preset templates remain available if results are poor. AI never writes any text on the card.

Exports. Excel sheets are grouped by department and sorted by year level, section and name, with one column per day and slot. PDFs use a print stylesheet so the dashboard prints cleanly.

Hosting, cost and performance targets

I have not checked current prices or limits, so verify them before you commit.

Component	Cost risk
Supabase	The free plan has caps on database size, storage and bandwidth, and may pause inactive projects. Event pack downloads are the main bandwidth use
Static hosting	Free tiers are usually enough for an app shell
AI image API	Charged per image, which is why generations are limited per event
Domain name	Optional; a free subdomain works for a pilot

Performance targets (proposed)

- Scan to verified result on the device: under 300 ms; scan to accepted screen: about 2 seconds including the SBO's glance.
- QR visible within 1 second of opening the event (location off).
- Roster pack for the largest department: 10 to 25 MB, downloaded once per event.
- Event pack per student: under 150 KB.
- Sync uploads in batches of about 500 records per request.

Testing plan and technical risks

Tests before any real event

Test	How	Passes when
Offline run	Airplane mode on both phones, full flow from QR to stamp	Everything works with no connection
Scan speed	Three phones, about 100 test students, stopwatch	Under 2 seconds each
Device matrix	Installed PWA on Safari (iOS 16.4 or newer) and Chrome on low-end Android	Camera, QR, storage and sync all work
Clock drift	Set a phone 2 minutes off	Clear "clock is off" message, no false accept
Duplicate scans	Two offline scanners scan the same student, then sync	Earliest kept, later flagged
Key loss	Clear the app's storage	The reset flow restores access
Forged card	Edit a card image to add signatures	Check card shows the slots as missing
Access rules	Sign in as a student and query other students	Refused by row-level security
Location	Walk a real venue, indoors and outdoors	Flag behavior matches the radius

Technical risks

Risk	Mitigation
iPhone may clear storage and lose the device key	Request persistent storage, tell officers to open the scanner the day before, keep the reset flow simple
Cheap phone cameras scan slowly	Large high-contrast QR, short payload, manual check-in fallback
Slow GPS cold start	Open the QR early, flag-only by default, easy to turn off
Free-tier limits hit at event time	Compress artwork, cache packs, check limits, budget for a paid plan
Stamp not on the student's card at once	Sync by default, optional receipt QR
Non-extractable keys unsupported on some browser	Test on target phones during Phase 0 before building on this design